TrayakshAI

NVIDIA AI Infrastructure, GPU Operations & Production Optimization

3-DAY ADVANCED TRAINING CURRICULUM

Infrastructure Architecture | Capacity Planning | Model Serving
Monitoring | Troubleshooting

Duration	3 Days
Format	Instructor-led theory with hands-on labs
Scope	14 Modules and 7 Labs

Curriculum for programme discussion and customization

1. Training Overview

This three-day course focuses on practical NVIDIA-aligned AI infrastructure and operations, including GPU architecture, capacity planning, production inference, developer–infrastructure collaboration, observability, resource optimization, cluster operations and incident troubleshooting. The curriculum retains a 14-module and 7-lab structure, while keeping general AI foundation topics concise to prioritize GPU operations, CUDA and driver architecture, inference performance, KV-cache behaviour, multi-GPU communication, monitoring and production troubleshooting.

2. Learning Outcomes

- Explain the end-to-end GPU software stack, from hardware, firmware and drivers through CUDA libraries, frameworks, runtimes and applications.
- Size GPU memory for LLM workloads using model parameters, precision, KV cache, context length, concurrency and runtime overhead.
- Deploy and operate open-weight models using vLLM and explain PagedAttention, continuous batching, request queues and KV-cache management.
- Measure production inference using time to first token, inter-token latency, tokens per second, throughput, concurrency and GPU memory utilization.
- Monitor GPU infrastructure using nvidia-smi, NVML, DCGM, Prometheus/Grafana, PyTorch telemetry and vLLM metrics.
- Troubleshoot CUDA out-of-memory events, Xid and ECC errors, driver failures, hung kernels, thermal or power throttling, interconnect bottlenecks and context exhaustion.
- Explain Kubernetes and Slurm orchestration, GPU operator components, MIG, time-slicing, governance and production recovery practices.

3. Detailed Curriculum

Day 1: AI Foundations, NVIDIA Architecture and GPU Capacity Planning

Module 1: Introduction to Artificial Intelligence and Generative AI Systems

- Artificial Intelligence and Generative AI fundamentals
- Concise evolution of AI technologies and key industry adoption patterns
- Representative enterprise AI use cases
- Infrastructure impacts of Generative AI and Large Language Models

Module 2: Generative AI and Emerging AI Applications

- Overview of LLMs, diffusion models and multimodal AI
- How generative models create content and manage token generation
- Enterprise deployment considerations for generative models
- Compute, memory, storage, networking and operational requirements
- LLM inference concepts for infrastructure operations

Module 3: NVIDIA AI Platform, GPU Architecture and CUDA Software Stack

- NVIDIA AI ecosystem: DGX systems, HGX platform and SuperPOD concepts
- CPU versus GPU architecture and parallel processing fundamentals
- Streaming Multiprocessors, CUDA cores, Tensor Cores and accelerated matrix computation
- GPU firmware and NVIDIA driver responsibilities

- Kernel driver versus user-space libraries and driver/runtime compatibility
- CUDA Driver API, CUDA Runtime API, CUDA Toolkit and NVCC overview
- Operational roles of cuBLAS, cuDNN, NCCL and NVML
- End-to-end software flow: application to framework/runtime, CUDA, driver and GPU
- Compute, storage and networking architecture for AI infrastructure pipelines

Lab 1: Exploring GPU Environments and the CUDA Software Stack
Objective: Access GPU resources; verify driver and CUDA runtime environments; use nvidia-smi; inspect loaded driver modules, GPU properties, compute capability and topology.

Module 4: GPU Workload Capacity Planning and Sizing

- Methodologies to assess infrastructure capacity against workload requirements
- LLM VRAM sizing using parameter count and bytes per parameter
- Precision impact: FP32, FP16/BF16, INT8, FP8 and quantized deployment
- KV-cache sizing using layers, KV heads, head dimension, precision, tokens and concurrency
- Impact of context window, batch, concurrency, model weights, activations, workspace and runtime overhead
- Assessment of whether existing GPUs can support additional workloads
- Optimization techniques before adding capacity
- Infrastructure–developer collaboration on model and runtime requirements

Lab 2: GPU Workload Sizing and Capacity Assessment
Objective: Calculate VRAM and compute requirements for target LLM and deep-learning workloads, then assess deployment feasibility under fixed hardware constraints.

Module 5: Storage, Networking and Multi-GPU Communication

- High-throughput, low-latency storage for AI pipelines
- Parallel file systems, NVMe-oF and object storage
- Networking requirements for multi-GPU and multi-node clusters
- PCIe topology and GPU locality
- NVLink and NVSwitch connectivity
- InfiniBand and RoCE for scale-out networking
- NCCL collective communication, topology awareness and bottlenecks

Day 2: Model Serving, KV Cache, Concurrency and Developer Optimization

Module 6: High-Performance Model Serving with vLLM

- Transition from training to inference and production serving
- Open-weight model loading and runtime memory allocation
- vLLM architecture: PagedAttention, continuous batching and KV-cache management
- KV-cache block allocation, reclamation and context-window configuration
- Runtime configuration, caching mechanisms and operational workflows
- Balancing context length, batching, concurrency and VRAM limits

Lab 3: vLLM Deployment, Configuration and Serving Walkthrough
Objective: Deploy an open-weight LLM using vLLM, configure model and context settings, observe weight and KV-cache memory, and inspect runtime behaviour under traffic.

Module 7: Concurrent User, KV Cache and Inference Performance Management

- GPU resource consumption by concurrent users and sequences
- Context windows, KV-cache growth, memory pressure and request queuing
- Continuous batching and parallel request execution
- Concurrency controls and service-level limits
- Metrics: TTFT, inter-token latency, tokens per second, throughput and latency percentiles
- Latency versus throughput trade-offs and SLA-oriented capacity decisions
- Diagnosis of compute, capacity, bandwidth, queueing and interconnect bottlenecks

Lab 4: Simulating Concurrent Workloads and VRAM Pressure
Objective: Generate multi-user requests, increase context and concurrency, monitor KV-cache and VRAM growth, observe queuing, and compare TTFT and throughput.

Module 8: GPU Best Practices for Databricks Environments

- GPU utilization and management in Databricks environments
- Selection of driver nodes, worker nodes and instance types
- Multi-GPU configurations, cluster policies and resource isolation
- Avoidance of unnecessary GPU consumption in notebook sessions

Module 9: PyTorch Architecture and Development Best Practices

- PyTorch GPU execution architecture and infrastructure-impacting practices
- VRAM optimization with automatic mixed precision, FP16/BF16 and INT8
- DataLoader optimization using pin_memory, num_workers and non-blocking transfers
- GPU memory allocation, caching and avoidance of unnecessary memory retention
- Developer-visible versus infrastructure-visible memory behaviour
- Basic profiling and telemetry for inefficient GPU use

Lab 5: PyTorch GPU Memory Optimization and Profiling
Objective: Profile a training or inference script, identify unnecessary VRAM usage or leaks, apply mixed precision and transfer optimizations, and compare memory behaviour.

Day 3: GPU Monitoring, Cluster Operations and Production Troubleshooting

Module 10: Maximizing NVIDIA Hardware Utilization

- Techniques to maximize GPU performance and utilization
- Tensor Cores, NVLink, NVSwitch and TensorRT
- MIG partitioning versus time-slicing for workload isolation
- Power caps, thermals, throttling and energy efficiency
- Why high utilization does not always indicate strong application throughput

Module 11: Dual-Persona GPU Monitoring: Infrastructure and Developer

- Infrastructure metrics: utilization, memory, temperature, power, PCIe, ECC and cluster health
- Relationship between NVIDIA driver, NVML and nvidia-smi or monitoring applications
- DCGM and dcgmi fundamentals; DCGM Exporter with Prometheus and Grafana
- Developer telemetry: PyTorch memory tracking, kernel profiling and vLLM metrics
- Tool-based versus code-based monitoring
- Correlation of infrastructure metrics with TTFT, latency, throughput and queue depth

Lab 6: Co-Creating Reusable Monitoring and Utility Scripts
Objective: Create reusable Python utilities using pynvml and PyTorch memory APIs for utilization, power and thermal telemetry, alerting and workload diagnostics.

Module 12: AI Infrastructure Operations and Cluster Orchestration

- Provisioning, allocation, scheduling and lifecycle management
- Kubernetes and Slurm orchestration concepts
- NVIDIA GPU Operator architecture and component responsibilities
- Driver management, container toolkit, device plugin, GPU Feature Discovery, DCGM Exporter and MIG Manager
- Resource isolation, automation, optimization and fault detection

Module 13: Real-World Operational Scenarios and Incident Management

- CUDA out-of-memory diagnosis across weights, KV cache and runtime memory
- Hung CUDA kernels, stuck GPU processes and recovery decisions
- NVIDIA Xid errors and software-versus-hardware evidence paths
- ECC single-bit and double-bit errors and response
- Driver, CUDA and runtime compatibility troubleshooting
- Thermal throttling and power-related degradation
- PCIe, NVLink, NVSwitch and NCCL bottlenecks
- Context exhaustion, queue saturation and inference degradation
- Cross-functional workflows between infrastructure and data science teams
- Production triage, workload recovery, graceful degradation and escalation

Lab 7: Production Incident and Workload Recovery Simulation
Objective: Diagnose controlled failures such as CUDA OOM, leaks, stuck tasks, context exhaustion, degraded latency, GPU health errors and interconnect issues; identify evidence, likely cause and recovery action.

Module 14: Use Cases, Deployment Patterns and Best Practices

- On-premises, hybrid and cloud deployment patterns
- GPU asset governance and cost optimization
- Production readiness for capacity, observability, isolation, recovery and change management
- Packaging reusable scripts and operational templates

4. Hands-on Lab Environment and Preparation Requirements

The seven labs progressively reuse a common prepared GPU environment. A separate GPU server is not required for each lab. vLLM, PyTorch, monitoring utilities and controlled troubleshooting scenarios are introduced as the programme progresses.

Lab	Preparation Requirement
Lab 1 – GPU Environment and CUDA Stack	Direct GPU access, NVIDIA driver, CUDA environment, nvidia-smi and topology visibility.
Lab 2 – GPU Workload Sizing	Same GPU server; sizing exercises using model parameters, precision, KV cache, context, concurrency and runtime overhead.
Lab 3 – vLLM Deployment	vLLM and PyTorch installed, with a small open-weight model pre-downloaded.
Lab 4 – Concurrent Workloads	Reachable vLLM API and a workload generator for simultaneous requests.
Lab 5 – PyTorch Optimization	PyTorch with CUDA and a deliberately inefficient training or inference script.
Lab 6 – Monitoring	Python monitoring packages including pynvml; where practical, DCGM Exporter, Prometheus and Grafana.
Lab 7 – Troubleshooting and Recovery	Controlled failure scenarios for OOM, excessive VRAM, stuck tasks, context exhaustion, latency degradation and recovery.

5. Recommended Common Lab Build

GPU Server → NVIDIA Driver/CUDA → Python/PyTorch → vLLM + Open-Weight Model → API Workload Generator → pynvml → Optional DCGM/Prometheus/Grafana

6. Final Production Troubleshooting Challenge

Participants follow a structured workflow: Symptom → Metric → Command/Tool → Infrastructure Layer → Likely Cause → Corrective Action.

- High GPU utilization with low tokens per second
- Increasing TTFT as concurrent users rise
- CUDA OOM during longer prompts
- A missing or unavailable GPU
- Xid or ECC health events
- Degraded NVLink or NCCL communication
- Thermal or power throttling
- vLLM request queue or KV-cache pressure causing SLA degradation

7. Recommended Three-Day Delivery Balance

Day	Primary Focus	Theory / Discussion	Hands-on
Day 1	Architecture and Capacity	~5 hours	~2 hours
Day 2	Inference, Concurrency and PyTorch	~4.5–5 hours	~2–2.5 hours
Day 3	Monitoring, Operations and Troubleshooting	~4.5–5 hours	~2–2.5 hours